MAINAYA – VIRTUAL APPOINTMENT MANAGEMENT SYSTEM

Project Documentation

Abstract

MAINAYA – Virtual Appointment Management System is a web-based application developed to simplify appointment scheduling between students and coordinators in an academic institution. The system enables students to register, log in, submit appointment requests, and monitor their appointment status. Coordinators can review requests, approve or reject appointments, and suggest alternative schedules when necessary. Built using HTML, CSS, JavaScript, and Firebase, the system provides secure authentication, role-based access control, real-time data management, and an intuitive user interface. MAINAYA enhances communication, reduces manual scheduling efforts, and ensures efficient appointment management.

Nithin S. Shetty

nithinsshetty3@gmail.com

Contents

Laboratory 1

Title of the Laboratory Exercise: Requirements Analysis - I

1. **Introduction and Purpose of Experiment**
    - Our team analyzed the given scenario, "***Virtual Appointment System***," to understand how to document and define software requirements. This experiment helped us learn the basics of requirement analysis and prepare for further stages of project development.

2. **Aim and Objectives**

    Aim
    - To prepare clear and structured software requirements for the given problem, *"Virtual Appointment System."*

    Objectives
    - By completing this lab, our team was able to:
        - Understand the user needs and expectations from the system.
        - Determine the scope and limitations of the project.
        - Develop a foundation for designing and implementing the system effectively.

3. **Experimental Procedure**
    - Our team of three members reviewed the problem statement, *"Virtual Appointment System."*
    - We discussed and identified all necessary software requirements together.
    - The finalized requirements were documented in a Software Requirements Specification (SRS) format.
    - Each member then prepared their individual lab manual, recording observations and outcomes of the exercise.

4. **Calculations/Computations/Algorithms**

    For the Virtual Appointment System, the focus is on the system workflow and logic rather than numerical calculations. The requirements and algorithm for the system are as follows:

    Functional Requirements (FR):
    - **FR1**: The system should allow new students and coordinators to register by entering required details.
    - **FR2**: The system should allow registered students and coordinators to login using valid credentials.
    - **FR3**: Students can submit appointment requests specifying purpose, coordinator, date, and time.
    - **FR4**: Coordinators can view all pending appointment requests submitted by students.
    - **FR5**: Coordinators can approve submitted student appointment requests.
    - **FR6**: Coordinators can reject student requests and provide an alternative suggested date and time.
    - **FR7**: The system automatically displays students regarding approval or rejection.
    - **FR8**: Students can view the updated status (Pending / Approved / Rejected) in their portal.
    - **FR9**: Both students and coordinators can securely logout from the system.

Non-Functional Requirements (NFR):

- **NFR1** – Security: Ensure login credentials and user data are protected to prevent unauthorized access.
- **NFR2** – Usability: Provide a simple and intuitive user interface for easy navigation.
- **NFR3** – Performance: The system must respond quickly when submitting requests or updating appointment status.
- **NFR4** – Scalability: The system should handle multiple student requests and coordinator actions simultaneously.
- **NFR5** – Maintainability: The system should be easy to update, modify, and fix issues in future versions.

Algorithm / Workflow:

- Start:
  User chooses to log in (Student / Coordinator)
- Student Workflow:
  Submit appointment request
  System stores request as Pending
- Coordinator Workflow:
  View pending requests
  Approve OR Reject with suggested date & time
- System Actions:
  Update status in student portal
  Send notification (Approval / Rejection)
- Logout
  End

5. **Analysis and Discussions**

In this lab, our team analyzed the Virtual Appointment System and organized its requirements into functional and non-functional categories.

- **Functional Requirements:**
  Students can log in, select the purpose and faculty, and submit appointment requests with preferred date and time. Coordinators can view pending requests, approve or reject them, and suggest alternative timings. The system also updates request status, sends displays, and supports secure logout.
- **Non-Functional Requirements:**
  These include security of user data, ease of use, quick system response, ability to handle many users, and simple maintenance.

Overall, this exercise helped our team understand how requirement analysis defines system behaviour and quality, forming a foundation for system design and development.

6. **Conclusions**

From this lab, our team learned how to identify and document the software requirements for the Virtual Appointment System. We analyzed the problem, classified the functional and non-functional requirements, and created a clear requirements list through teamwork and discussion. This will serve as a strong foundation for the design and development of the system.

Laboratory 2

Title of the Laboratory Exercise: Requirements Analysis - II

1) **Introduction and Purpose of Experiment**
    - In this laboratory exercise, we analyzed the requirements for the given problem statement and documented them in a formal SRS format. Our selected system was a Virtual Appointment Management System used by students and coordinators. The purpose of the experiment was to clearly identify, structure, and record the functional and non-functional requirements needed for the system.

2) **Aim and Objectives**

    Aim

    - To develop a formal SRS document in a standard format for the Virtual Appointment Management System.

    Objectives

    - Identify dependencies among software requirements.
    - Document requirements in a formal and structured SRS format.
    - Understand stakeholder roles in requirement definition.
    - Provide clear examples of user–system interactions.

3) **Experimental Procedure**
- We worked in a team of 3 members.
- Our team read and understood the problem statement.
- The functional and non-functional requirements were identified and finalized.
- Each requirement was documented formally in SRS format with:
    - ➢ Requirement tag
    - ➢ Requirement statement
    - ➢ Dependencies
    - ➢ Stakeholder ownership
    - ➢ Example of user/system interaction
- Each individual documented their observations in the lab manual

4) **Calculations/Computations/Algorithms**

Requirement tag	FR1
Requirement Statement	The system should allow new students and coordinators to register by entering required details.
Dependent on Requirements	-
Stake Holder Owning the Requirements	Students, Coordinators
Example of user/system interaction for this requirements	• Student should be able to register using studentID and password • Coordinator should be able to register using coordinatorID and password.

Requirement tag	FR2
Requirement Statement	The system should allow registered students and coordinators to log in using valid credentials.
Dependent on Requirements	FR1
Stake Holder Owning the Requirements	Students, Coordinators
Example of user/system interaction for this requirements	• Student should be able to log in using registered credentials. • Coordinator should be able to log in using coordinator login details.

Requirement tag	FR3
Requirement Statement	Students can submit appointment requests specifying purpose, coordinator, date, and time.
Dependent on Requirements	FR2
Stake Holder Owning the Requirements	Students
Example of user/system interaction for this requirements	• Student should be able to choose date, time, and purpose. • Student should be able to select a specific coordinator. • The system should store the request in "Pending" status.

Requirement tag	FR4
Requirement Statement	Coordinators can view all pending appointment requests submitted by students.
Dependent on Requirements	FR2, FR3
Stake Holder Owning the Requirements	Coordinators
Example of user/system interaction for this requirements	• Coordinator should be able to see all pending requests of their department. • Coordinator should sort requests based on date/time and their availability.

Requirement tag	FR5
Requirement Statement	Coordinators can approve submitted student appointment requests.
Dependent on Requirements	FR3, FR4
Stake Holder Owning the Requirements	Coordinators
Example of user/system interaction for this requirements	• Coordinator should approve Student X's request with a single click. • System updates the request status to "Approved."

Requirement tag	FR6
Requirement Statement	Coordinators can reject student requests and provide an alternative date and time..
Dependent on Requirements	FR3, FR4
Stake Holder Owning the Requirements	Coordinators
Example of user/system interaction for this requirements	• Coordinator should reject a request and suggest a new schedule. • System stores the alternative suggested date/time.

Requirement tag	FR7
Requirement Statement	The system automatically displays students regarding approval or rejection.
Dependent on Requirements	FR5, FR6
Stake Holder Owning the Requirements	Students, Coordinators, System Admin
Example of user/system interaction for this requirements	• Student can view their approved appointment. • Student can view message containing coordinator's suggested date and time.

Requirement tag	FR8
Requirement Statement	Students can view the updated status (Pending / Approved / Rejected) in their portal
Dependent on Requirements	FR3, FR5, FR6, FR7
Stake Holder Owning the Requirements	Students
Example of user/system interaction for this requirements	• Student should see real-time status updates in their dashboard. • Student should be able to view coordinator suggestions.

Requirement tag	FR9
Requirement Statement	Both students and coordinators can securely log out from the system.
Dependent on Requirements	FR2
Stake Holder Owning the Requirements	Students, Coordinators
Example of user/system interaction for this requirements	• Student should be able to log out after checking their status. • Coordinator should log out after responding to pending requests

5) Analysis and Discussions

- In this laboratory exercise, we formally documented and identified the requirements for the Virtual Appointment System.
  We developed an SRS containing all 9 functional requirements and 5 non-functional requirements.
- Each requirement was documented with dependencies, stakeholders, and real-life examples of user/system interaction.
  Through teamwork, we understood requirement flow, relationships, and system behaviour in detail.

6) Conclusions
   - We successfully documented the requirements for the Virtual Appointment System in a formal SRS format.
   - This exercise helped us understand requirement analysis, dependency mapping, and structured documentation as part of the software engineering process.

Laboratory 3

Title of the Laboratory Exercise: UML Modelling: Use Case Diagram

1. **Introduction and Purpose of Experiment**
    - In this experiment, our team applied UML modelling to create a Use Case diagram for the Virtual Appointment System. The goal was to understand how the system works through its users, interactions, and major functionalities in a visual manner.

2. **Aim and Objectives**

   Aim
    - To develop the high-level design for the Virtual Appointment System using UML Use Case modelling.

   Objectives
    - Identify the context and boundaries of the system
    - Identify actors (Students and Coordinators) and their interactions
    - Recognize all functional requirements as use cases
    - Understand relationships such as association, include, and exclude
    - Create a structured design document for the system

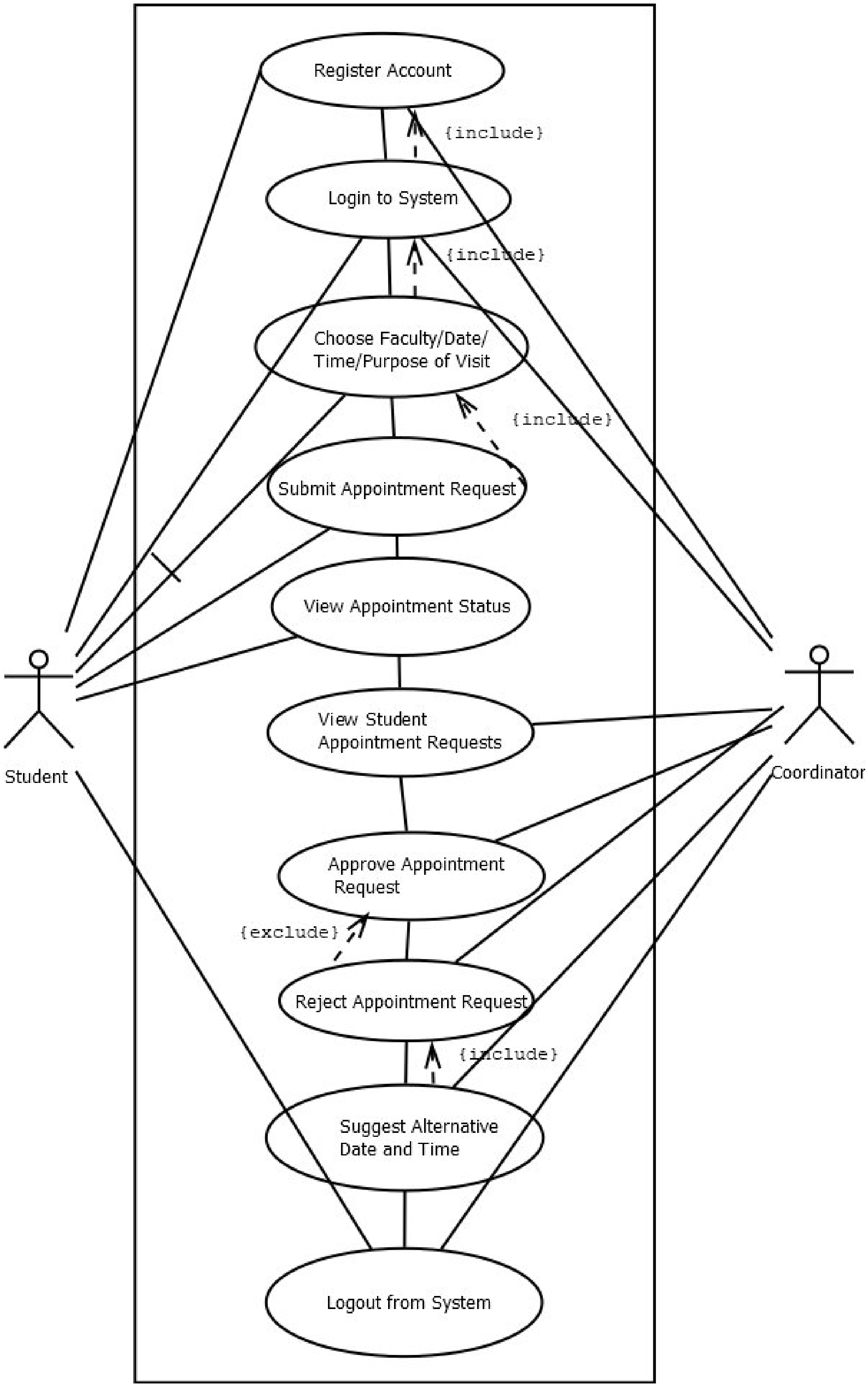
3. **Experimental Procedure**
    - Our team of 3 students reviewed the problem statement and identified all functional requirements.
    - We selected the major functionalities to be represented as use cases.
    - We identified the main actors: Student and Coordinator.
    - We used DIA to draw the Use Case diagram.
    - We connected actors and use cases using associations and dependencies.
    - Each team member then documented the results and observations individually.

4. **Calculations/Computations/Algorithms**

Step 1: Convert each functional requirement of the Virtual Appointment System into a Use Case (e.g., Login, Submit Request, Approve Request, Display , Logout).
Step 2: Identify all actors: Student and Coordinator
Step 3: Add the main relationships:
- Association between actors and use cases
- <<exclude>> for rejection with suggested time
- <<include>> for signup and login

Step 4: Build the complete Use Case diagram
Step 5: Ensure actors only connect to use cases, and dependencies connect between use cases.
Step 6: Export the diagram as PNG for final documentation.

5. Presentation of Results



6. Analysis and Discussions

- In this lab, we learned how to convert the functional requirements of the Virtual Appointment System into a structured UML Use Case diagram.

7. Conclusions

- We successfully applied UML modelling to design the high-level Use Case diagram for the Virtual Appointment System this helped us clearly understand how users interact with the system and how the system supports every functionality, which prepares us for detailed system design and implementation.

Laboratory 4

Title of the Laboratory Exercise: UML Modelling: Class Diagrams

1. **Introduction and Purpose of Experiment**
   In this laboratory exercise, our team applied **object-oriented design (OOD)** for the *Virtual Appointment Management System*.
   The purpose of the experiment was to decompose the system into classes, identify their attributes and methods, and determine appropriate relationships.
   This helped us understand the structural design of the system using a **UML Class Diagram**.

2. **Aim and Objectives**
   Aim
   - To construct a UML Class Diagram for the Virtual Appointment System and identify class members, attributes, methods, and relationships.

   Objectives

   - Identify the main classes involved in the appointment system
   - Determine how the classes are related to each other
   - Identify the attributes and operations of each class
   - Determine associations, multiplicities, and inheritance relationships
   - Apply object-oriented principles to decompose the system into meaningful class structures

3. **Experimental Procedure**
   - We worked in teams of **three members**.
   - The team reviewed all previously identified functional requirements from earlier labs.
   - The main system components were identified and converted into classes.
   - Attributes and operations were assigned to each class based on system functionalities.
   - Relationships such as association and generalization were identified.
   - The UML Class Diagram was drawn using DIA software.
   - Each member documented the class diagram design and observations individually in their lab manual.

4. **Calculations/Computations/Algorithms**

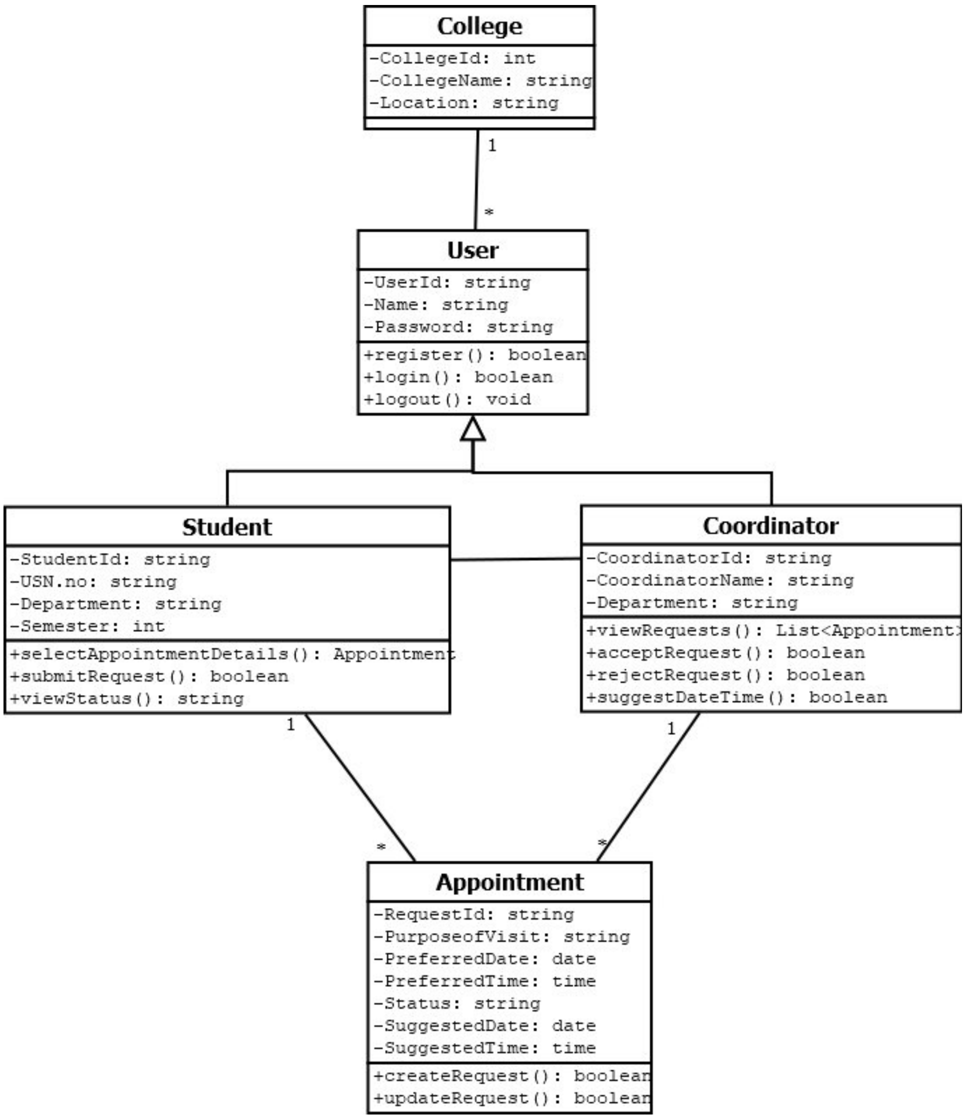
   - Step 1: Identify Classes
     Classes identified for the Virtual Token / Appointment System:
     User (Base Class)
     Student (Inherited Class)
     Coordinator (Inherited Class)
     AppointmentRequest
     Notification
     SystemAdmin

- **Step 2**: Identify Relationships
  Generalization (Inheritance):
  Student -> User
  Coordinator -> User
  Association:
  Student submits AppointmentRequest
  Coordinator reviews AppointmentRequest

- Step 3: Assign Attributes and Methods
  Each class is given a name, attributes and methods

- Step 4: Apply Multiplicity/Cardinality
  One Student can submit 1..* AppointmentRequests
  One Coordinator can handle 1..* AppointmentRequests
- Step 5: Export Diagram
  The completed UML Class Diagram is saved as a PNG file for documentation.

5. Presentation of Results



6. Analysis and Discussions

- In this lab, we learned how to analyze requirements and convert them into a structural OO model using a UML Class Diagram.
  We identified major system entities such as User, Student, Coordinator & AppointmentRequest.
- Using DIA software, we assigned appropriate attributes and operations functions to each class.
  We determined inheritance relationships, associations, and multiplicities based on system behavior.

7. Conclusions

- We successfully designed and modelled the Class Diagram for the Virtual Appointment System.
- We learned how to apply object-oriented principles for object decomposition and how to visually represent system structure using UML tools.
- We identified class members, relationships, inheritance patterns, and associations, forming a strong foundation for further system design and implementation.

Laboratory 5

Title of the Laboratory Exercise: UML Modelling: State Chart Diagram

1. **Introduction and Purpose of Experiment**
    - In this lab, our team modelled the dynamic behaviour of the Virtual Appointment System using UML State Chart Diagram.
      We studied how objects change states based on events and triggers, helping us understand the system's low-level behaviour.

2. **Aim and Objectives**

    Aim

    - To develop low level software design for a given class diagram using state chart diagrams

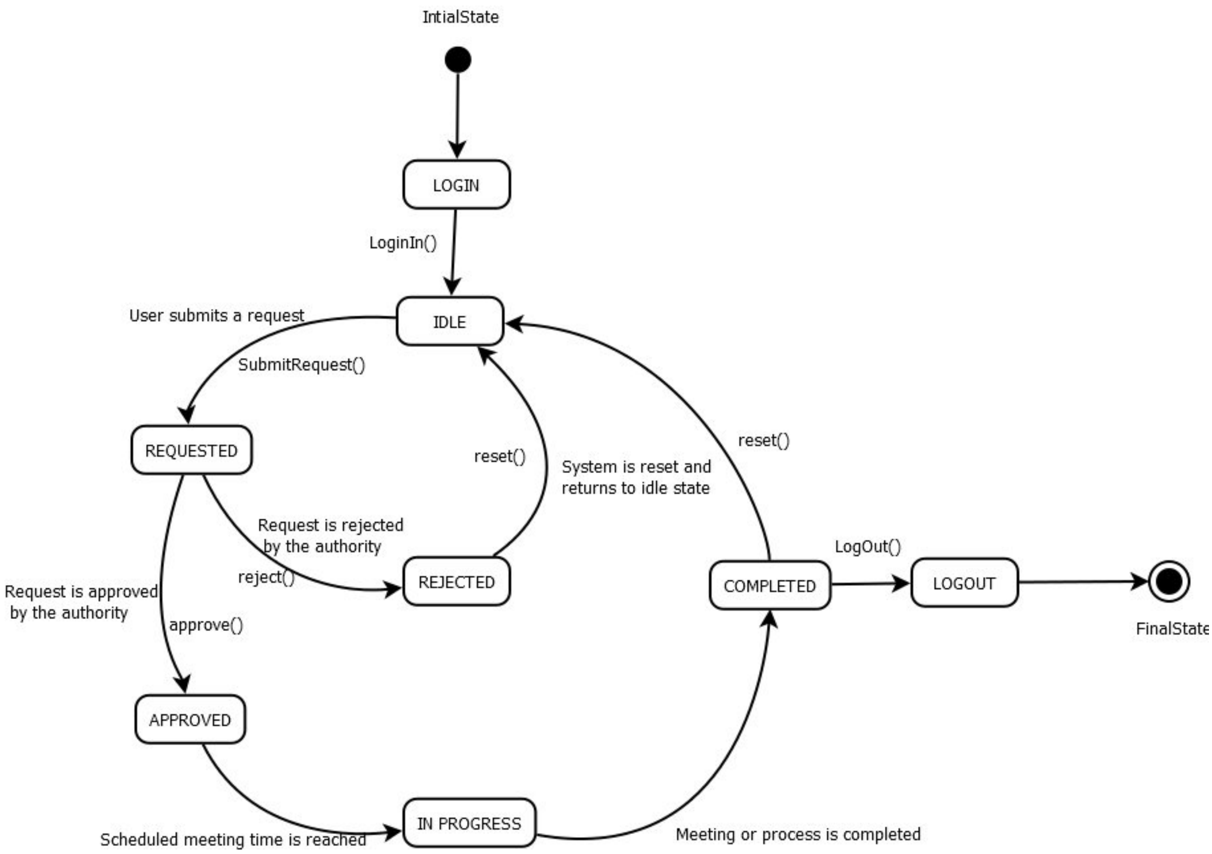
    Objectives

    At the end of this lab, our team was able to:

    - Identify states of each object
    - Identify triggers and messages for each object
    - Understand the behavior of a class, given its state chart diagram

3. **Experimental Procedure**
    - We worked in teams of 3 members.
    - The team reviewed the Virtual Appointment System flow and identified all states involved during user interaction.
    - We identified the key states such as Idle, Registration, Login, Student Dashboard, Coordinator Dashboard, and Logout.
    - Triggers and actions for each state (entry, do, exit) were listed based on system behaviour.
    - Using DIA software, we created a single State Chart Diagram representing the entire user flow of the system.
    - The transitions were checked to ensure correct movement between states.
    - Each member documented their observations individually.

4. Presentation of Results



5. Analysis and Discussions

- We analysed how the user of the Virtual Appointment System transitions between states such as Idle, Registration, Login, Student, Coordinator, and Logout.
- The State Chart Diagram clearly shows how events like login, submit request, accept, reject, and logout drive the transitions.
- The diagram helped us understand the dynamic behaviour of the system from both the student's and coordinator's perspective.
- This experiment strengthened our understanding of modelling event-driven state changes

6. Conclusions

- We successfully created a State Chart Diagram representing the complete user flow of the Virtual Appointment System, including Registration, Login, Student operations, Coordinator actions, and Logout.
- The experiment helped our team understand how states, triggers, and transitions control the system’s dynamic behaviour.
- We gained deeper knowledge of modelling event-driven behaviour and visualizing how users interact with the system at a low level.
- This documentation will support further design tasks such as activity diagrams and implementation.

Laboratory 6

Title of the Laboratory Exercise: UML Modelling: Activity Diagrams

1. **Introduction and Purpose of Experiment**
    - In this laboratory exercise, students apply object-oriented analysis and design techniques to model the **Virtual Appointment System** using **UML Activity Diagrams**. The purpose of this experiment is to perform **low-level system design** by representing workflows, actions, and control flows involved in the system.
2. **Aim and Objectives**

Aim

- To develop low-level software design for a given system using **Activity Diagrams**.

Objectives

At the end of this lab, our team was able to:

- Identify activities involved in the system
- Identify decision points, fork and join nodes
- Understand the behaviour of the system using an activity diagram

3. **Experimental Procedure**
    - We worked in teams of 3 members.
    - The team studied the system scenario and reviewed documents such as SRS, use case diagrams, class diagrams, and state chart diagrams.
    - Activities, interactions, and workflows of the **Student** and **Coordinator** were identified.
    - Control flows, decisions, and parallel activities were analysed.
    - The **Activity Diagram** was designed and simulated using **DIA software**.
    - The low-level design was documented.
    - Each individual documented their observations in the lab manual.

4. **Calculations / Computations / Algorithms**

Algorithm for Activity Diagram Construction

step 1 Start the diagram using the initial node.

step 2 Identify functional requirements of the Virtual Appointment System and represent them as action states.

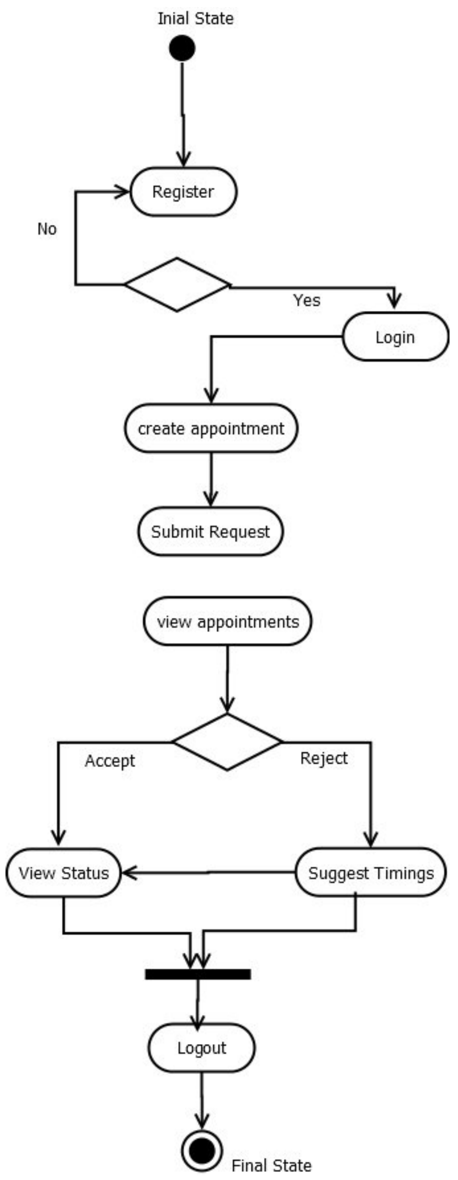
step 3 Draw transitions between action states using directed arrows.

- Use decision nodes for conditional flows
- Use fork nodes for parallel activities
- Use join nodes to merge parallel flows

step 4 End the diagram using the final node.

step 5 Save the activity diagram as a PNG image for documentation.

5. **Presentation of Results**



6. Analysis and Discussions

In this experiment, the Virtual Appointment System was modelled using an Activity Diagram.

The activity diagram helped in understanding:

- The flow of activities from login to logout
- Decision-making processes such as approval and rejection
- Parallel execution of tasks like saving requests and updating dashboards

Activity diagrams clearly represent the behavioural flow of the system and show how control moves between different actions

7. Conclusions

The Activity Diagram for the Virtual Appointment System was successfully developed.

This experiment helped in understanding how activity diagrams are used for:

- Low-level system design
- Modelling dynamic behaviour
- Representing parallel and sequential flows

Laboratory 7

Title of the Laboratory Exercise: UML Modelling: Data Flow Diagrams (DFD)

1. **Introduction and Purpose of Experiment**
    - In this laboratory exercise, students apply object-oriented analysis and design concepts to model the **Virtual Appointment System** using **Data Flow Diagrams (DFD)**.
      The purpose of this experiment is to perform **low-level system design** by representing the flow of data between external entities, processes, and data stores within the system.

2. **Aim and Objectives**

   Aim

    - To develop low-level software design for a given system using **Data Flow Diagrams**.

   Objectives

   At the end of this lab, our team was able to:

    - Identify external entities involved in the system
    - Identify data flows between processes
    - Understand he functional behaviour of the system using DFDs

3. **Experimental Procedure**
    - We worked in teams of 3 members.
    - The team studied the system scenario and identified external entities such as **Student** and **Coordinator**.
    - System processes, data flows, and data stores were analysed.
    - A **Level-0 (Context Diagram)** was designed to represent the system as a single process.
    - The context diagram was decomposed into **Level-1 DFD** to show major system processes.
    - Further decomposition was carried out to obtain a **Level-2 DFD** showing detailed subprocesses.
    - All diagrams were verified for correctness and consistency.
    - Each individual documented their observations in the lab manual.

4. **Calculations / Computations / Algorithms**

Steps for Constructing Data Flow Diagrams

step 1 Design the Level-0 DFD (Context Diagram) representing the Virtual Appointment System as a single process with input and output data flows from external entities.

step 2 Develop the Level-1 DFD by decomposing the context diagram into major processes such as:

- Registration Process
- Login Process
- Book Appointment
- View Appointment Requests
- Accept / Reject Appointment

step 3 Develop the Level-2 DFD by further decomposing Level-1 processes into detailed subprocesses such as:

- Capture registration details
- Verify username and password
- Select coordinator, date, and time
- Approve or reject appointment
- Update appointment status

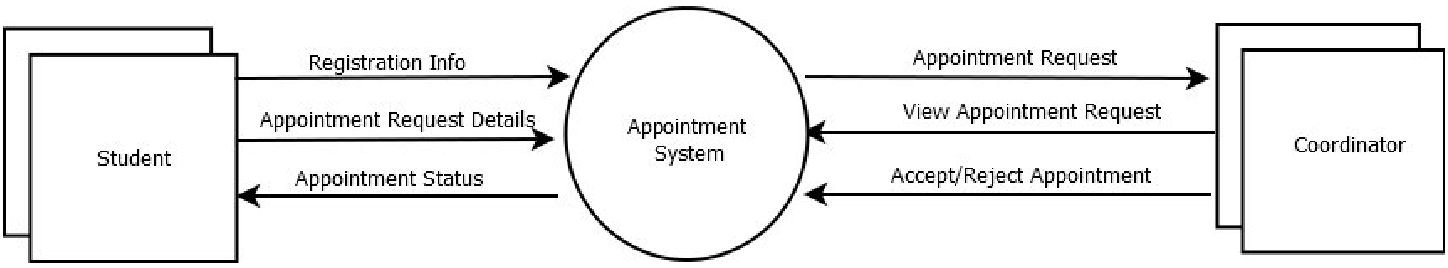
step 4 Identify data stores such as:

- User Data Storage
- Student Data Storage
- Coordinator Data Storage
- Appointment Data Storage

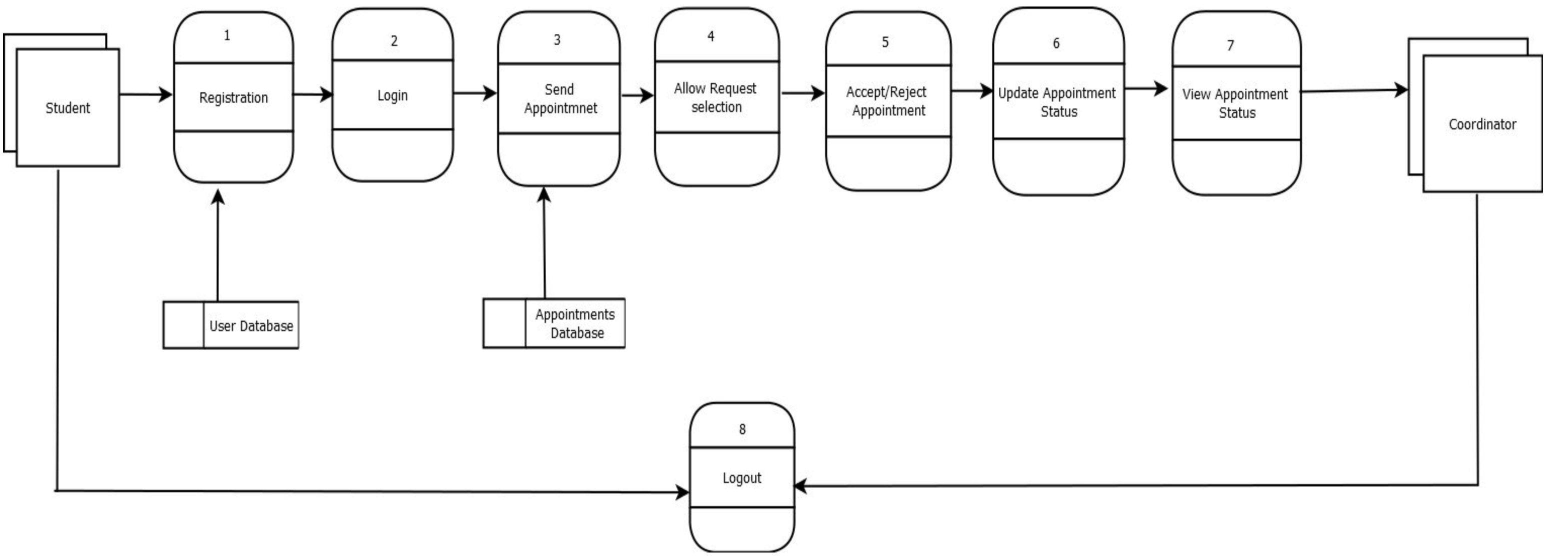
step 5 Save all DFD diagrams as PNG images for documentation.

5. Presentation of Results

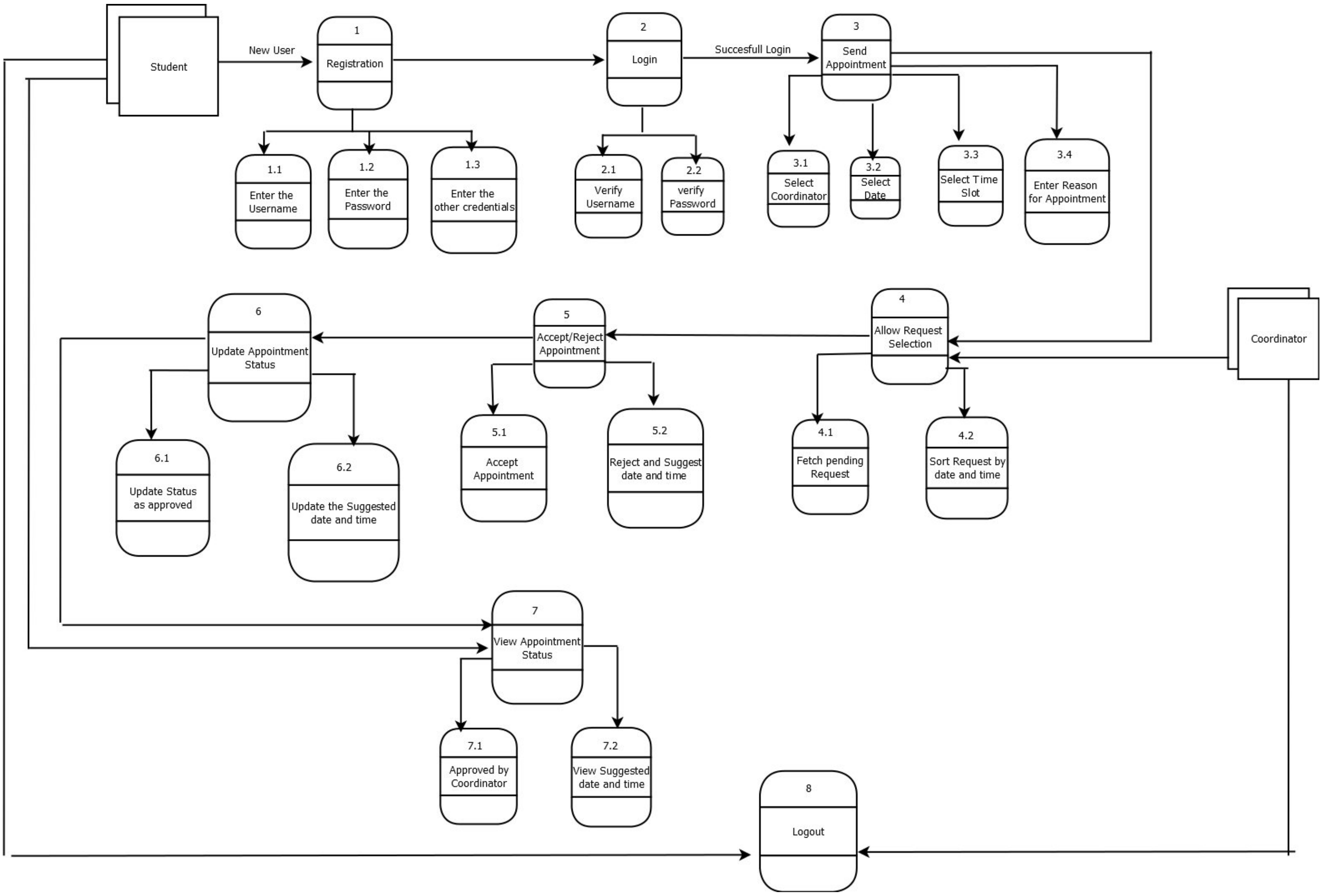
LEVEL-0



LEVEL-1



LEVEL-2



6. Analysis and Discussions

In this experiment, the **Virtual Appointment System** was modelled using **Data Flow Diagrams**. The DFDs clearly represent:

- Flow of data between Student and Coordinator
- Internal processing of appointment requests
- Storage and retrieval of user and appointment data
- Hierarchical decomposition of system functionality

DFDs helped in understanding how data moves through the system and how each process transforms input data into output data.

7. Conclusions

The Data Flow Diagrams for the Virtual Appointment System were successfully developed. This experiment helped in understanding:

- Functional decomposition of a system
- Representation of data movement and processing
- Use of hierarchical DFD levels for detailed system modelling

Laboratory 8

Title of the Laboratory Exercise: Implementation of Software Design for Virtual Appointment Management System

1) **Introduction and Purpose of Experiment**
    - In this laboratory experiment, students apply object-oriented analysis concepts to implement the Virtual Appointment Management System. The purpose of this experiment is to convert the requirements identified in earlier laboratories into a working software system by identifying appropriate classes, attributes, and interactions.

2) **Aim and Objectives**

a. Aim
    - To develop the source code for the requirements specified in Laboratory 1 and Laboratory 2.

b. Objectives

At the end of this lab, the team was able to:
    - Identify the different classes involved in the system
    - Identify and implement different variables for storing user and appointment data

3) **Experimental Procedure**

- Worked in teams of 3 students
- Each team studied the requirements specified in **Lab 1 and Lab 2**
- Identified system components such as users, appointments, and roles
- Implemented the system based on the identified design
- Each individual documented observations and implementation details in the lab manual

4) **Website Link**

- **https://mainaya.web.app/**

5) **Introduction of Virtual Appointment Management System**

The Virtual Appointment Management System is designed to facilitate appointment scheduling between Students and Coordinators in an academic environment.
A Not Registered User must first register by providing valid details such as name, email ID, department, and password. Once registered, the Registered User can log in using valid credentials.
After successful login:

- A Student can submit an appointment request by selecting the coordinator, specifying the purpose, preferred date, and time.
- The appointment request is stored with a Pending status.
- The Coordinator can view all appointment requests assigned to them.
- The Coordinator has the authority to approve an appointment or reject it by suggesting an alternative date and time.
- The student can view the updated appointment status (Approved / Rejected / Pending) along with any suggested changes.

The system also provides:

- Appointment status tracking
- Secure logout functionality
- Controlled access based on user roles

The Virtual Appointment Management System improves communication efficiency, reduces manual scheduling conflicts, and ensures structured appointment handling.

6) **Implementation:**

- LOGIN

```
<body>

<div class="page" id="loginPage">
  <div class="center-card">
    <h2>Login</h2>

    <input id="loginEmail" type="email" placeholder="Email">
    <input id="loginPass" type="password" placeholder="Password">

    <button onclick="login()">Login</button>
  </div>
</div>

<script>
  async function login(){
    const email = document.getElementById("loginEmail").value.trim();
    const password = document.getElementById("loginPass").value.trim();

    if(!email || !password){
      alert("Please enter email and password");
      return;
    }

    try{
      await firebase.auth()
        .signInWithEmailAndPassword(email, password);

      alert("Login Successful");
    } catch(error){
      alert("Login Failed: " + error.message);
    }
  }
</script>

</body>
</html>
```

- STUDENT APPOINTMENT BOOKING

```
<body>

<div class="page">
  <div class="card">
    <h3>Book Appointment</h3>

    <input id="purpose" type="text" placeholder="Purpose of Appointment">
    <select id="faculty">
      <option value="">Select Coordinator</option>
    </select>
    <input id="date" type="date">
    <input id="time" type="time">

    <button onclick="submitAppointment()">Submit Appointment</button>
  </div>
</div>

```

```
<script>
  const db = firebase.firestore();
  const auth = firebase.auth();

  // Load coordinators into dropdown
  db.collection("users")
    .where("role", "==", "coordinator")
    .onSnapshot(snapshot => {
      const faculty = document.getElementById("faculty");
      faculty.innerHTML = '<option value="">Select Coordinator</option>';
      snapshot.forEach(doc => {
        const data = doc.data();
        const option = document.createElement("option");
        option.value = doc.id;
        option.textContent = data.name + " (" + data.dept + ")";
        faculty.appendChild(option);
      });
    });

  async function submitAppointment(){
    const purpose = document.getElementById("purpose").value.trim();
    const facultyUid = document.getElementById("faculty").value;
    const date = document.getElementById("date").value;
    const time = document.getElementById("time").value;

    if(!purpose || !facultyUid || !date || !time){
      alert("Please fill all fields");
      return;
    }

    await db.collection("appointments").add({
      studentUid: auth.currentUser.uid,
      purpose: purpose,
      facultyUid: facultyUid,
      date: date,
      time: time,
      status: "Pending",
      createdAt: firebase.firestore.FieldValue.serverTimestamp()
    });

    alert("Appointment Request Submitted");

    document.getElementById("purpose").value = "";
    document.getElementById("faculty").selectedIndex = 0;
    document.getElementById("date").value = "";
    document.getElementById("time").value = "";
  }
</script>
```

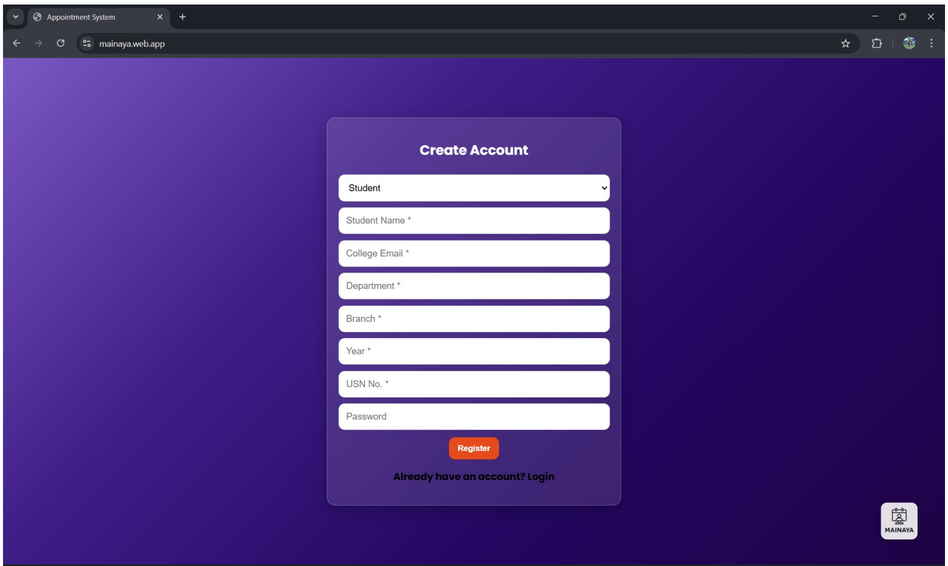
- COORDINATOR PAGE

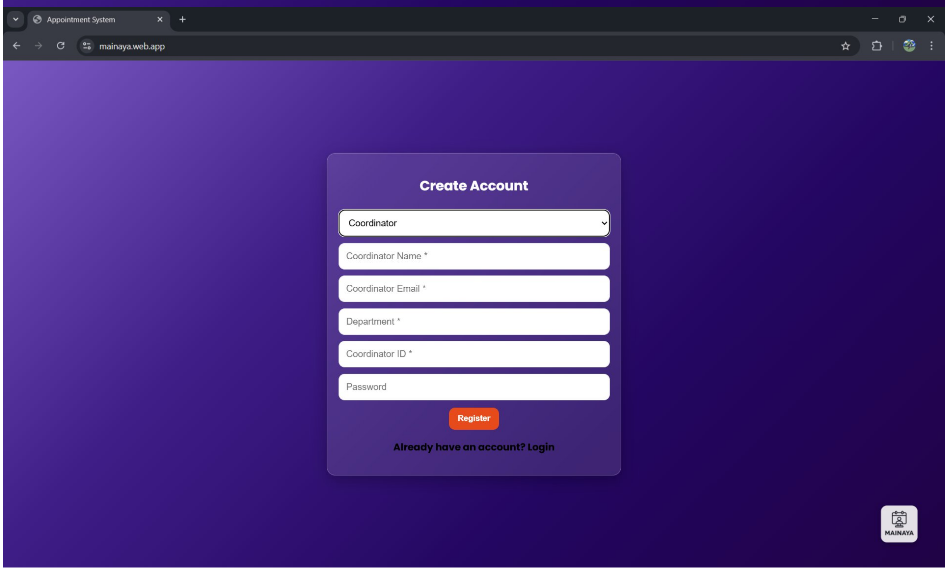
```

  // Approve request
  async function approveRequest(id){
    await db.collection("appointments")
      .doc(id)
      .update({ status: "Approved" });

    alert("Appointment Approved");
  }

  // Ask for rejection details
  function rejectPrompt(id){
    selectedRequestId = id;
    const date = prompt("Enter new suggested date (YYYY-MM-DD):");
    const time = prompt("Enter new suggested time (HH:MM):");

    if(date && time){
      submitRejection(date, time);
    }
  }

  // Reject with suggestion
  async function submitRejection(date, time){
    await db.collection("appointments")
      .doc(selectedRequestId)
      .update({
        status: "Rejected",
        suggestedDate: date,
        suggestedTime: time
      });

    alert("Appointment Rejected with Suggestion");
  }
</script>

</body>
</html>

```

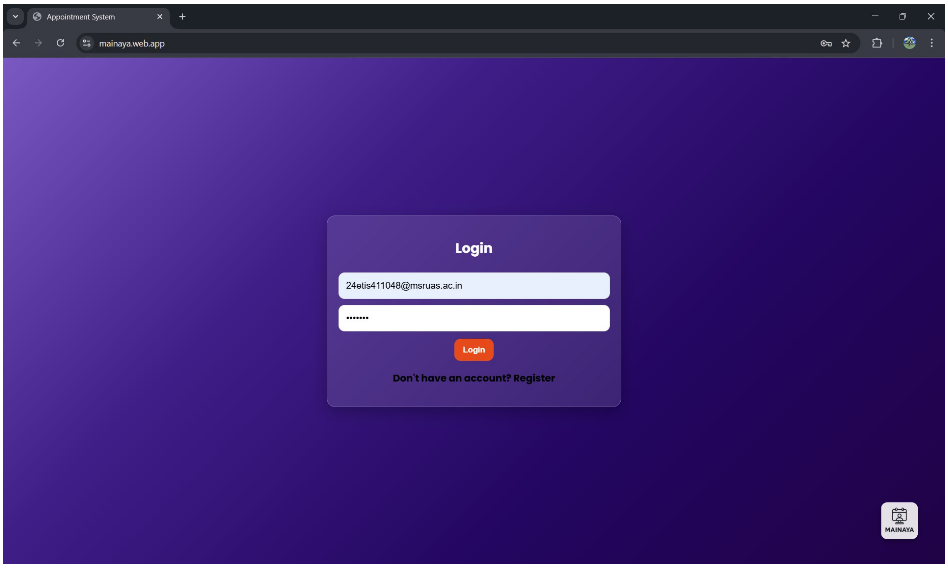
- APPOINTMENT STATUS

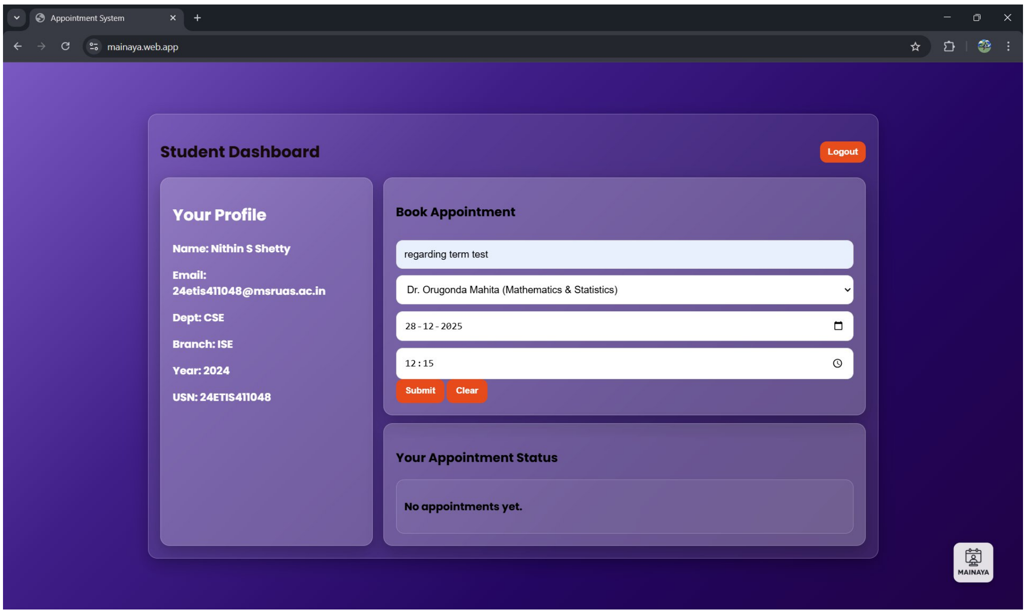
```
<body>

<div class="container">
  <h3>Your Appointment Status</h3>
  <div id="statusList"></div>
</div>

<script>
  const db = firebase.firestore();
  const auth = firebase.auth();

  auth.onAuthStateChanged(user => {
    if(!user) return;

    db.collection("appointments")
      .where("studentUid", "==", user.uid)
      .orderBy("createdAt", "desc")
      .onSnapshot(snapshot => {
        const list = document.getElementById("statusList");
        list.innerHTML = "";

        if(snapshot.empty){
          list.innerHTML = "<p>No appointments found</p>";
          return;
        }

        snapshot.forEach(doc => {
          const r = doc.data();
          let statusClass = r.status.toLowerCase();

          list.innerHTML += `
            <div class="card">
              <p><b>Coordinator:</b> ${r.facultyName || "Coordinator"}</p>
              <p><b>Purpose:</b> ${r.purpose}</p>
              <p><b>Date:</b> ${r.date}   <b>Time:</b> ${r.time}</p>
              <p class="status ${statusClass}">
                Status: ${r.status}
              </p>

              ${
                r.suggestedDate && r.suggestedTime
                  ? `<p class="suggest">
                      Suggested: ${r.suggestedDate} at ${r.suggestedTime}
                    </p>`
                  : ""
              }
            </div>
          `;
        });
      });
  });
</script>

</body>
</html>
```

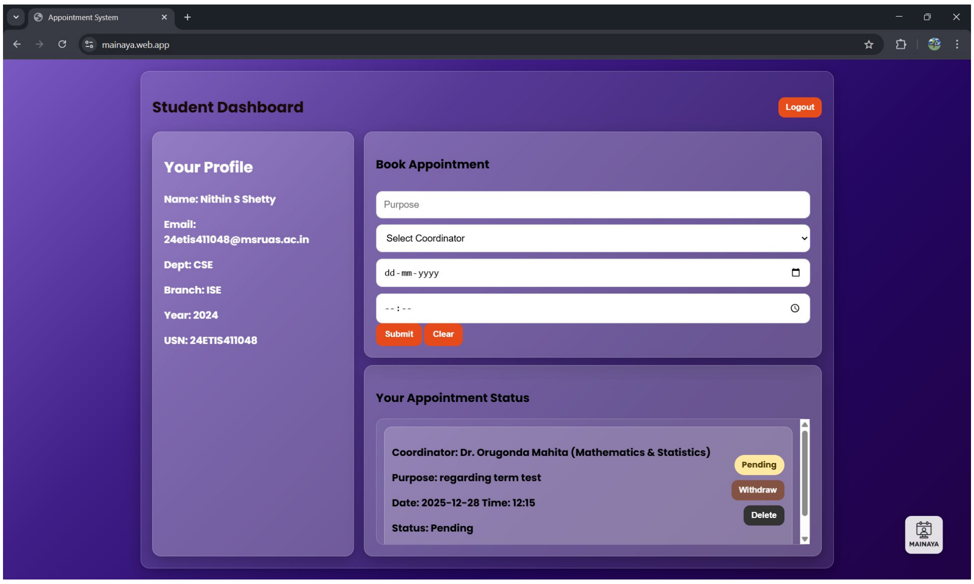
7) Results/Output of Software Program:

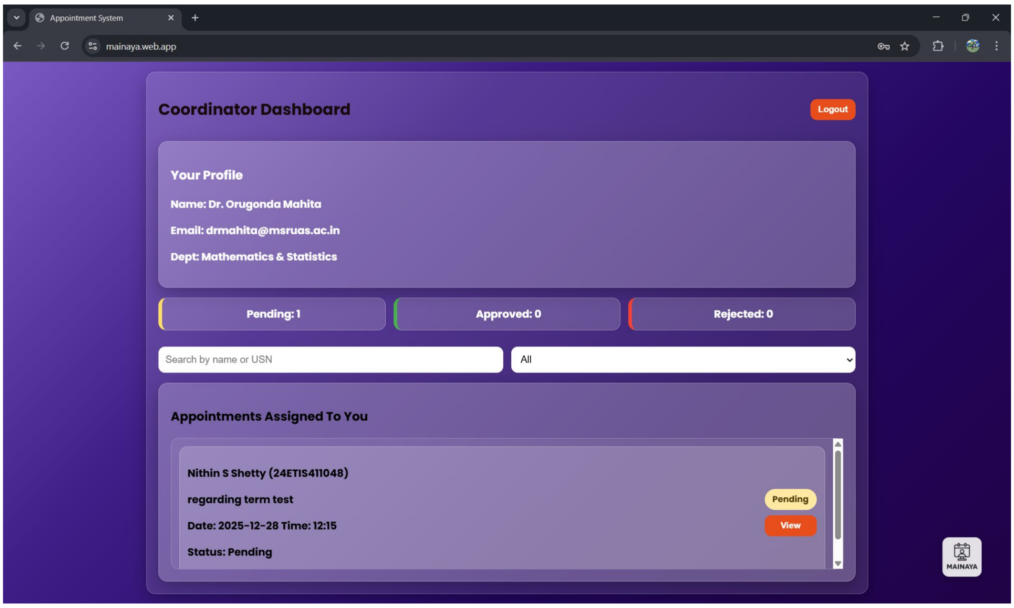


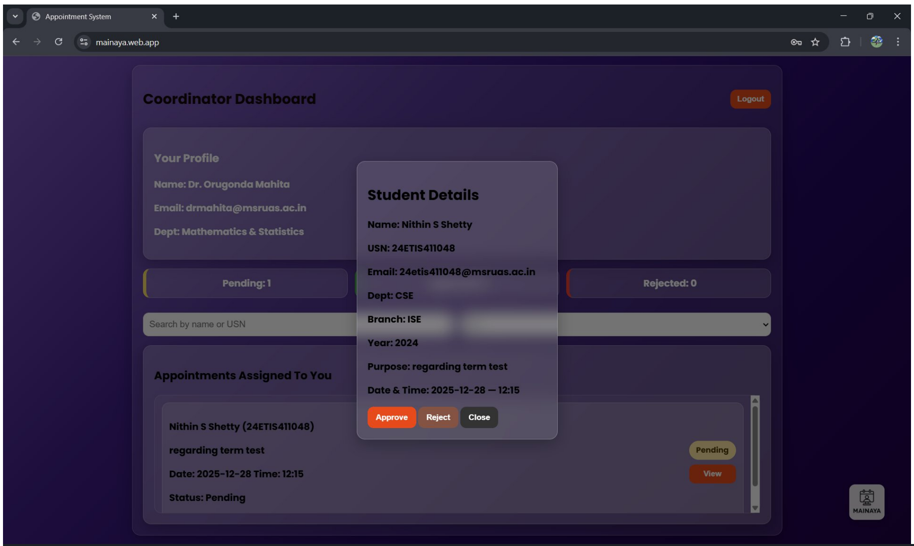


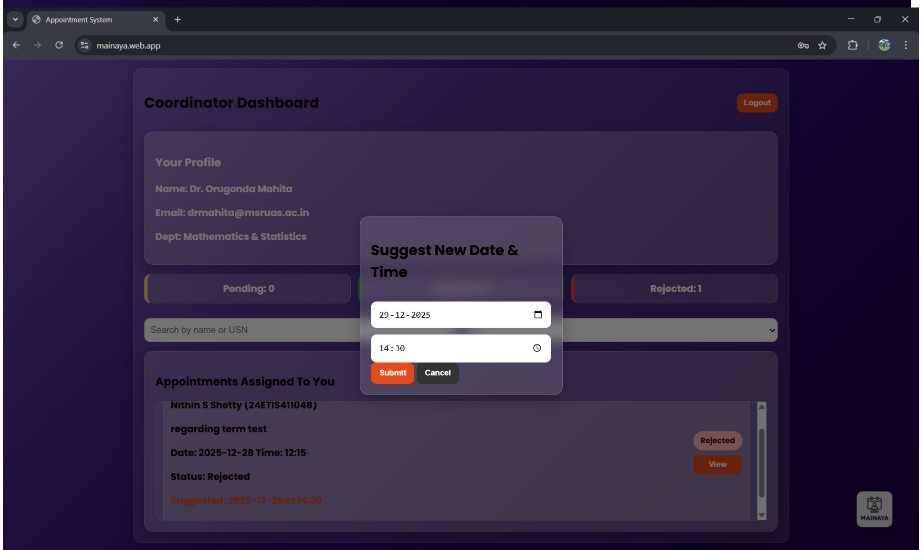


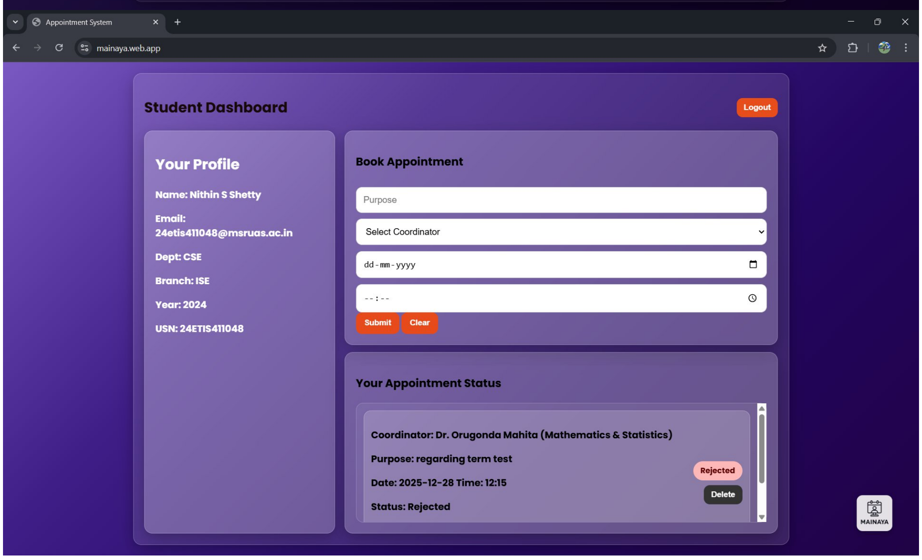

Appointment System
mainaya.web.app
Student Dashboard
Logout
Your Profile
Name: Nithin S Shetty
Email: 24etis411048@msruas.ac.in
Dept: CSE
Branch: ISE
Year: 2024
USN: 24ETIS411048
Book Appointment
regarding term test
Dr. Orugonda Mahita (Mathematics & Statistics)
28-12-2025
12:15
Submit
Clear
Your Appointment Status
No appointments yet.
MAINAYA


Appointment System
mainaya.web.app
Student Dashboard
Logout
Your Profile
Name: Nithin S Shetty
Email: 24etis411048@msruas.ac.in
Dept: CSE
Branch: ISE
Year: 2024
USN: 24ETIS411048
Book Appointment
Purpose
Select Coordinator
dd-mm-yyyy
--:--
Submit
Clear
Your Appointment Status
Coordinator: Dr. Orugonda Mahita (Mathematics & Statistics)
Purpose: regarding term test
Date: 2025-12-28 Time: 12:15
Status: Pending
Pending
Withdraw
Delete
MAINAYA


Appointment System
mainaya.web.app
Coordinator Dashboard
Logout
Your Profile
Name: Dr. Orugonda Mahita
Email: drmahita@msruas.ac.in
Dept: Mathematics & Statistics
Pending: 1
Approved: 0
Rejected: 0
Search by name or USN
All
Appointments Assigned To You
Nithin S Shetty (24ETIS411048)
regarding term test
Date: 2025-12-28 Time: 12:15
Status: Pending
Pending
View
MAINAYA







8) Analysis and Discussion

In this laboratory, the Virtual Appointment Management System was implemented using object-oriented principles based on requirements from earlier labs. Classes, attributes, and interactions were identified to support functions such as login, appointment booking, approval, rejection, and status tracking. The system workflow was analyzed to ensure correct role-based access and data handling.

9) Conclusion

The Virtual Appointment Management System was successfully implemented according to the specified requirements. The system provides efficient appointment scheduling, status management, and secure access, improving coordination between students and coordinators.

Laboratory 9

Title of the Laboratory Exercise: **Test Case Design and Testing**

1) Introduction and Purpose of Experiment
    - In this laboratory experiment, students apply test case design techniques to develop and execute test cases for the Virtual Appointment Management System. The purpose is to validate that the software meets all functional requirements and behaves correctly under different conditions..

2) Aim and Objectives

a. Aim
    - To develop test cases and test the software based on defined requirements.

b. Objectives

At the end of this lab, the team was able to:
- Identify appropriate test data for each functional requirement
- Test the software against the designed test cases
- Verify correct system behavior for all requirements

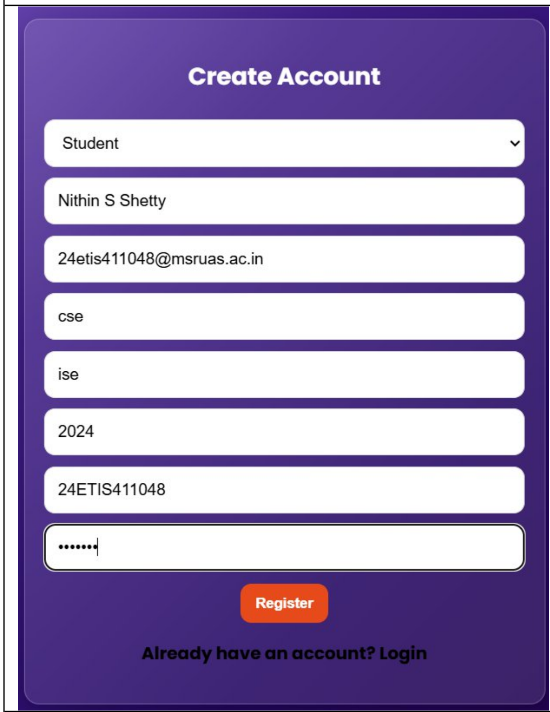
3) Experimental Procedure

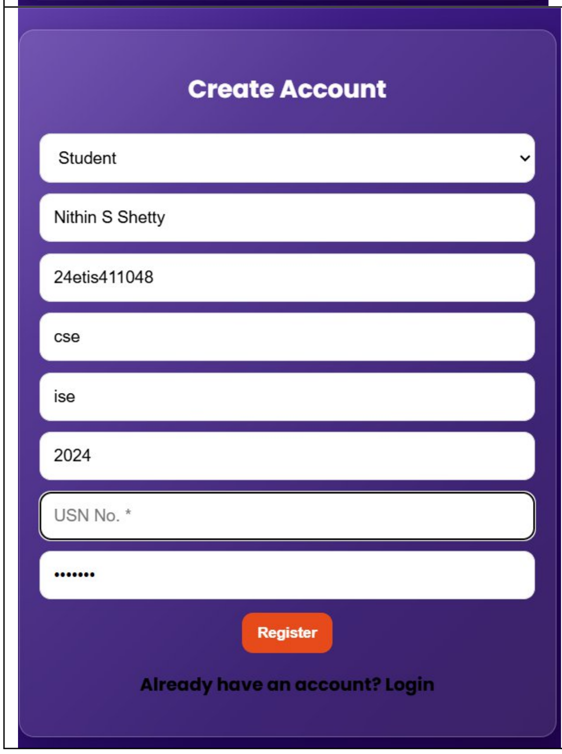
- Worked in teams of 3 students
- Each team developed test cases for different system scenarios
- Test cases were mapped to functional requirements
- The software was tested using the designed test cases
- Each individual documented observations and test results in the lab manual

4) Test Cases and Testing of the Virtual Appointment Management System

TEST CASE ID	FUNCTIONAL REQUIREMENT NUMBER	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT
TC 1	FR 1	The system should allow new students and coordinators to register by entering valid user details such as Name, Email-ID, USN ID, and Password. **Test Steps:-** **1)** Launch the Virtual Appointment Management System and select the Register option. **2)** In the registration screen, enter a user name , a valid email address in the format example@mail.com, a password and other	Valid Data ++++++++++++++++++++ Name: Nithin S Shetty Email-ID: 24etis411048@msruas.ac.in USN Number: 24ETIS411048 Password: MAINAYA Invalid Data ++++++++++++++++++++ Name: Nithin S Shetty Email-ID: 24etis411048 USN Number: 24ETIS411048 Password: MAINAYA Name: Nithin S Shetty	“Registration Successful” message is displayed. “Invalid Email-ID!” message is displayed.

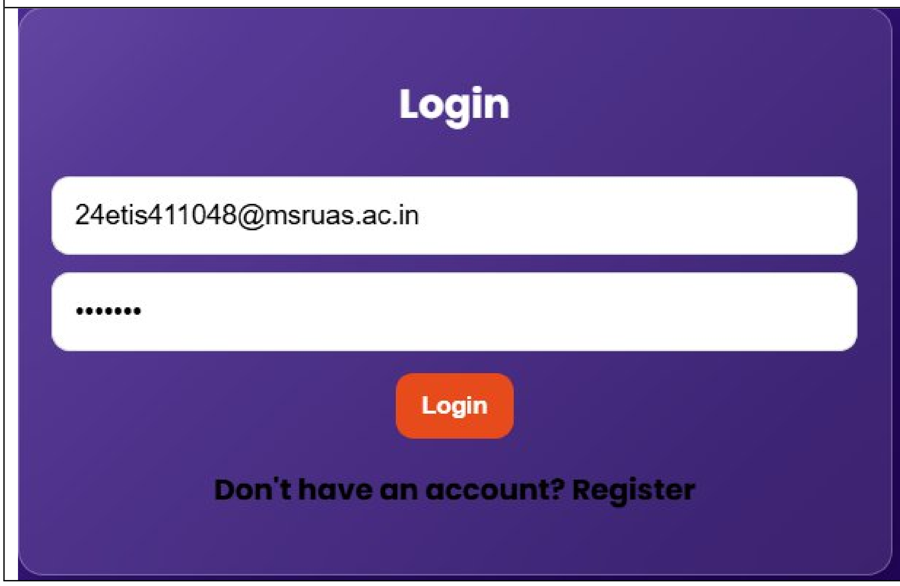
		details. **3)** Click on the Register button to complete the registration process.	Email-ID: 24etis411048@ruas.ac.in USN Number: Password: MAINAYA	"ENTER ALL THE FIELDS" message is displayed.

<table>
<tr><th>ACTUAL OUTPUT</th><th>TEST RESULY</th></tr>
<tr><td>


</td><td>pass</td></tr>
<tr><td>


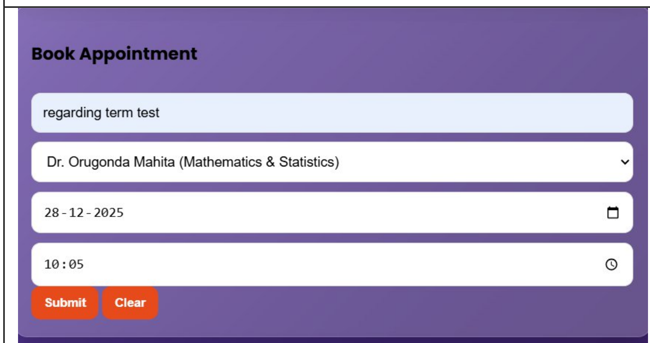
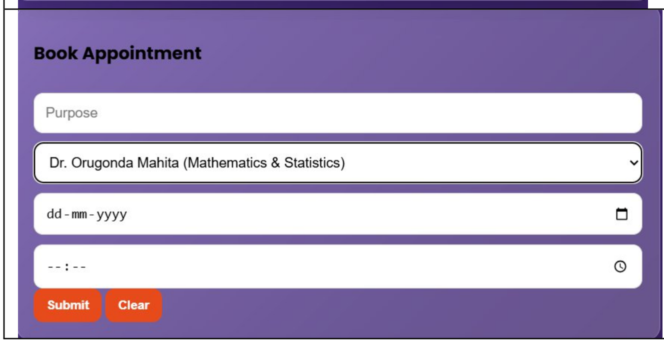
</td><td>fail</td></tr>
</table>

TEST CASE ID	FUNCTIONAL REQUIREMENT NUMBER	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT
TC 2	FR 2	The system should allow registered students and coordinators to login using valid credentials. Test Steps:- Step 1: Launch the Virtual Appointment Management System. Step 2: In the login screen, enter the registered email ID and password. Step 3: Click on the Login button to authenticate the user.	Valid Data ++++++++++++++++++++ Email-ID: 24ETIS411048@msruas.ac.in Password: (Valid registered password) Invalid Data – Case 1 ++++++++++++++++++++ Email-ID: 24ETIS411048@msruas.ac.in Password: WrongPassword Invalid Data – Case 2 ++++++++++++++++++++ Email-ID: nithin@msruas Password: AnyPassword	For valid credentials, user is successfully logged in and redirected to the respective dashboard (Student or Coordinator). "Invalid User Name or Password!" message is displayed. "Invalid User Name or Password!" message is displayed.

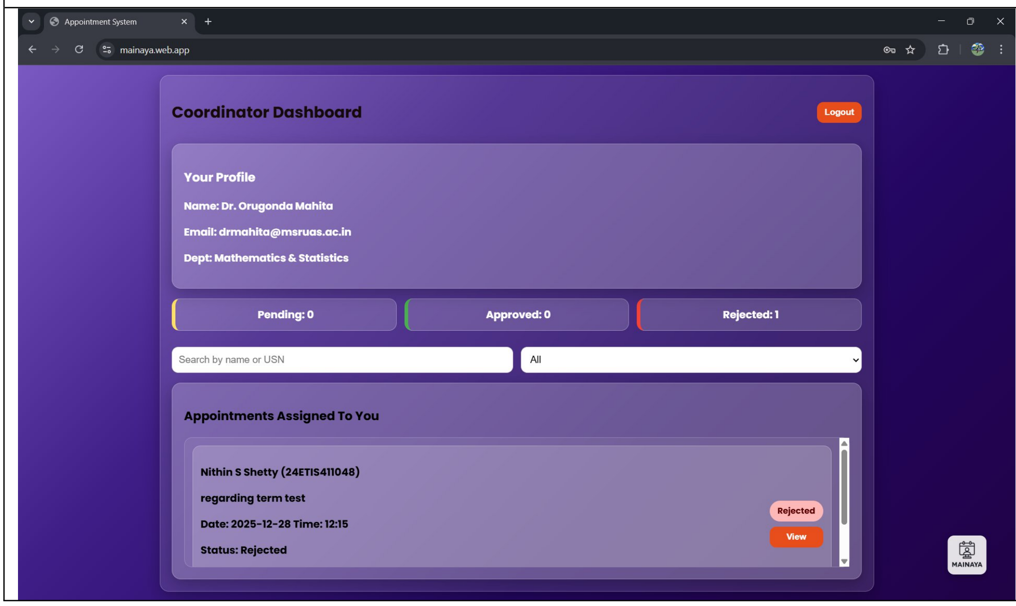
ACTUAL OUTPUT	TEST RESULT
	PASS

TEST	FUNCTIONAL	TEST CASE DESCRIPTION	TEST DATA	EXPECTED

CASE ID	REQUIREMENT NUMBER			OUTPUT
TC 3	FR 3	The system should allow students to submit an appointment request by specifying the purpose, coordinator, date, and time. Test Steps:- Step 1: Launch the Virtual Appointment Management System and login as a Student using valid credentials. Step 2: Navigate to the *Book Appointment* section in the student dashboard. Step 3: Enter the appointment purpose, select a coordinator, choose a valid date and time. Step 4: Click on the Submit Appointment button.	Valid Data ++++++++++++++++++++ Student Name: Nithin S Shetty Purpose: Project Discussion Coordinator: Dr. Ravi Kumar (CSE Department) Date: 15-03-2025 Time: 11:00 AM Invalid Data – Case 1 ++++++++++++++++++++ Purpose: *(Not entered)* Coordinator: Dr. Ravi Kumar Date: 15-03-2025 Time: 11:00 AM Invalid Data – Case 2 ++++++++++++++++++++ Purpose: Project Discussion Coordinator: *(Not selected)* Date: *(Not selected)* Time: *(Not selected)*	For valid input, appointment request is successfully submitted with status "Pending". If any mandatory field is missing, appropriate validation message is displayed and the request is not submitted.

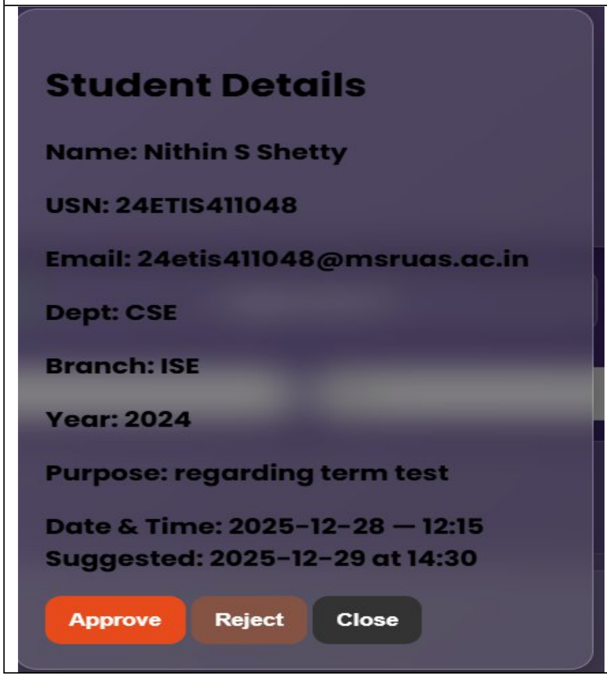
ACTUAL OUTPUT	TEST RESULT
	PASS
	FAIL

TEST CASE ID	FUNCTIONAL REQUIREMENT NUMBER	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT
TC 4	FR 4	The system should allow coordinators to view all pending appointment requests submitted by students. Test Steps:- Step 1: Launch the Virtual Appointment Management System. Step 2: Login as a Coordinator using valid credentials. Step 3: Navigate to the Coordinator Dashboard. Step 4: View the list of appointment requests.	Valid Data +++++++++++++++++++ Coordinator Name: Dr. Ravi Kumar Role: Coordinator Login Credentials: Valid coordinator email ID and password	A list of all pending appointment requests submitted by students is displayed on the coordinator dashboard.

<table>
<tr><th>ACTUAL OUTPUT</th><th>TEST RESULT</th></tr>
<tr><td>
</td><td>PASS</td></tr>
</table>

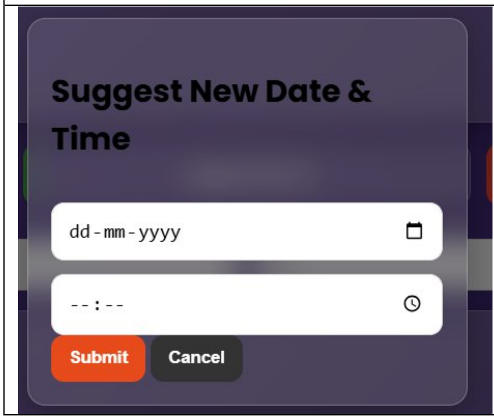
TEST CASE ID	FUNCTIONAL REQUIREMENT NUMBER	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT

TC5	FR 5	The system should allow coordinators to approve submitted student appointment requests. Test Steps:- Step 1: Launch the Virtual Appointment Management System. Step 2: Login as a Coordinator using valid credentials. Step 3: Navigate to the Coordinator Dashboard and view the list of pending appointment requests. Step 4: Select a pending appointment request and click on the Approve button. .	Valid Data ++++++++++++++++++++ Student Name: Nithin S Shetty USN: 24ETIS411048 Purpose: Regarding term test Requested Date & Time: 28-12-2025, 12:15 Appointment Status (Before Approval): Pending	The appointment request is approved successfully. The appointment status is updated to “Approved”. The updated status is visible to the student in the student portal.

ACTUAL OUTPUT	TEST RESULT
	PASS

TEST	FUNCTIONAL	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT

CASE ID	REQUIREMENT NUMBER			
TC 6	FR 6	The system should allow coordinators to reject student appointment requests and provide an alternative suggested date and time. Test Steps:- Step 1: Launch the Virtual Appointment Management System. Step 2: Login as a Coordinator using valid credentials. Step 3: Navigate to the Coordinator Dashboard and view the list of appointment requests. Step 4: Select a pending appointment request and click on the View button. Step 5: Click on the Reject button and enter a new suggested date and time. Step 6: Submit the rejection with the suggested schedule.	Valid Data ++++++++++++++++++++ Student Name: Nithin S Shetty USN: 24ETIS411048 Purpose: Regarding term test Original Date & Time: 28-12-2025, 12:15 Suggested Date: 29-12-2025 Suggested Time: 14:30	The appointment request is rejected successfully. The appointment status is updated to “Rejected”. The suggested date and time are displayed to the student in the student portal.

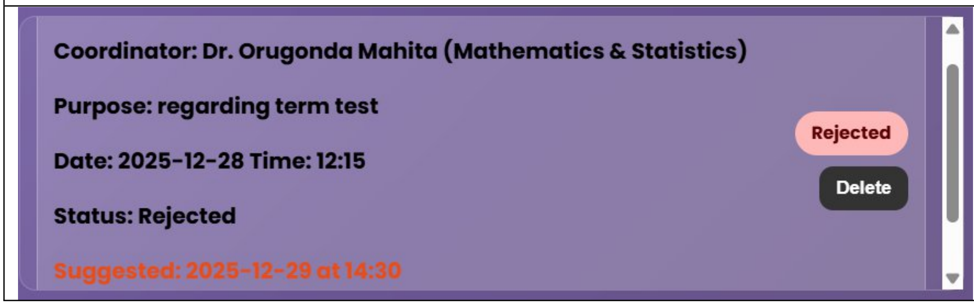
ACTUAL OUTPUT	TEST RESULT
	PASS

TEST	FUNCTIONAL	TEST CASE	TEST DATA	EXPECTED OUTPUT

CASE ID	REQUIREMENT NUMBER	DESCRIPTION		
TC 7	FR 7	The system should automatically notify students after their appointment request is approved or rejected by the coordinator. Test Steps:- Step 1: Launch the Virtual Appointment Management System. Step 2: Login as a Student using valid credentials. Step 3: Submit an appointment request to a coordinator. Step 4: Login as the Coordinator and approve or reject the submitted appointment request. Step 5: Login again as the Student and navigate to the student dashboard.	Valid Data ++++++++++++++++++++ Student Name: Nithin S Shetty USN: 24ETIS411048 Appointment Status: Approved / Rejected Suggested Date & Time (if rejected): 29-12-2025, 14:30	🎬 The student is automatically notified of the appointment decision. 🎬 The updated appointment status (Approved / Rejected) is displayed in the student portal. 🎬 If rejected, the suggested date and time are also shown to the student. Please Try Again!!"

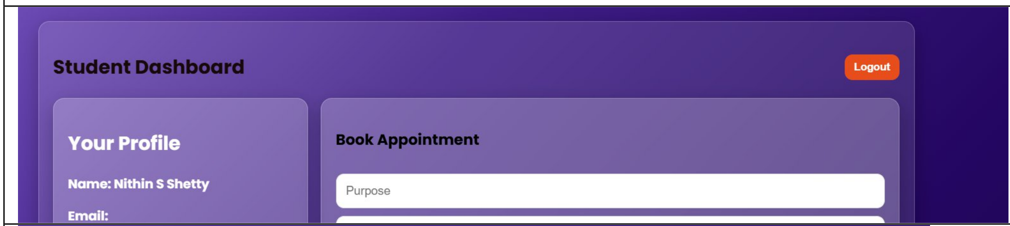
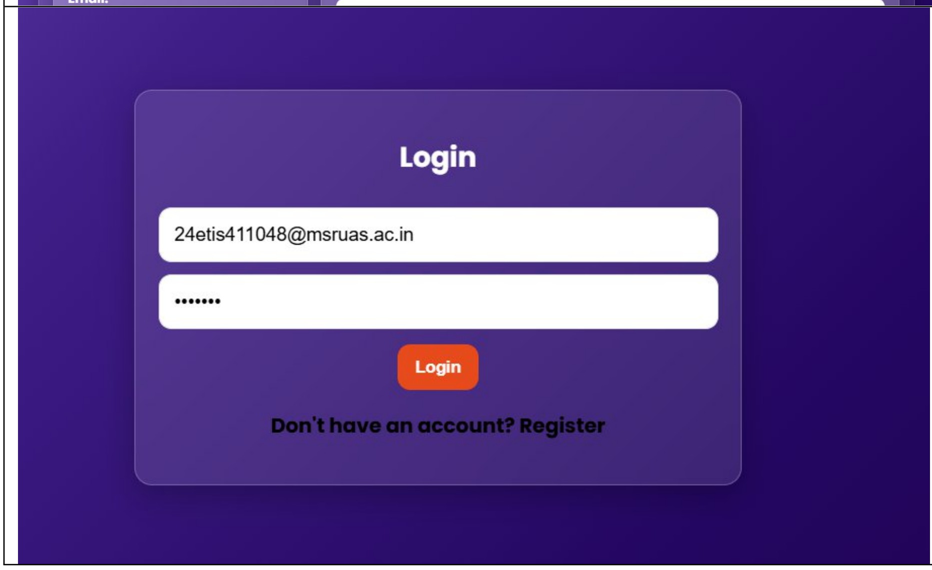
TEST CASE ID	FUNCTIONAL REQUIREMENT NUMBER	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT
TC 8	FR 8	Test Steps:- Step 1:	Valid Data	

		Launch the Virtual Appointment Management System. Step 2: Login as a Student using valid credentials. Step 3: Navigate to the Student Dashboard. Step 4: View the list of submitted appointment requests and their current status.	+++++++++++++++++++ Student Name: Nithin S Shetty USN: 24ETIS411048 Appointment Status: Pending / Approved / Rejected	The student dashboard displays the appointment request details. The appointment status is clearly shown as Pending, Approved, or Rejected. If rejected, the suggested date and time (if any) are also displayed.

ACTUAL OUTPUT	TEST RESULT
	PASS

TEST CASE ID	FUNCTIONAL REQUIREMENT NUMBER	TEST CASE DESCRIPTION	TEST DATA	EXPECTED OUTPUT
TC 9	FR 9	The system should allow students and coordinators to securely logout	Valid Data +++++++++++++++++++	The current user

		from the system. Test Steps:- Step 1: Launch the Virtual Appointment Management System. Step 2: Login as a Student or Coordinator using valid credentials. Step 3: Navigate to the respective dashboard. Step 4: Click on the Logout button.	User Name: Nithin S Shetty Role: Student / Coordinator Action: Logout	session is terminated successfully. The system redirects the user to the Login page. The user cannot access the dashboard without logging in again.

ACTUAL OUTPUT	TEST RESULT
	PASS
	PASS

5) **Analysis and Discussion**

In this laboratory, test cases were designed and executed for the Virtual Appointment Management System by mapping each functional requirement to appropriate test scenarios. Both valid and invalid test data were used to verify system behavior under different conditions. The execution of test cases helped identify correct functionality, validation handling, and role-based access control. The results confirmed that most system features worked as expected, while also highlighting areas requiring proper input validation.

6) **Conclusion**

In this laboratory exercise, we successfully designed and tested the Virtual Appointment Management System using well-defined test cases. All major functionalities such as registration, login, appointment booking, approval, rejection, status viewing, notification, and logout were verified. The testing process improved our understanding of test case design, requirement validation, and system reliability, ensuring that the software meets its specified requirements.